

Hospital Management Database - Data Generation & Schema Design Report

Student Name: Mahesh Babu Poka

Student ID: 24070896

1. Introduction

This project aims to design and generate a realistic multi-table relational database in SQLite, following a real-world domain. I selected **Hospital Patient Management**, as hospitals naturally involve complex relationships between patients, clinicians, encounters, diagnoses, procedures, medications, and billing.

All data in this database was **synthetically generated using Python**, with no downloaded datasets used at any point. Randomisation was applied carefully using defined rules and clinical ranges to ensure realism. The final database contains **8 interconnected tables** with **over 5,900 rows**, satisfying all assignment and rubric requirements.

2. Data Generation Methodology

The database was created programmatically using Python's sqlite3 library. A reproducible random seed (random.seed(42)) ensured consistent results. Separate generation functions were written to produce each component of the dataset:

- **Patient names** randomly combined from real first name lists
- **Sex, height, weight, blood groups** generated within medical norms
- **Encounter dates/times** generated between 2021–2025
- **Vital signs** generated using real-world ranges
- **Procedure and drug lists** curated manually to reflect typical hospital usage
- **Billing** computed using consultation fees + procedure costs + medication counts

Deliberate **NULL values** were inserted into emergency contacts, settlement dates, and some clinical fields to improve realism and satisfy the assessment requirement for realistic missingness.

3. Database Schema Overview

The final database consists of **eight interconnected tables**, each representing a different component of the hospital workflow:

Table Name	Purpose
care_units	Defines hospital departments such as Emergency and Cardiology
clinician_registry	Stores clinician identity, specialty, and unit mapping
client_profiles	Patient demographics and physiological attributes
clinical_encounters	Every visit to the hospital (date, time, vitals, triage)
diagnostic_events	Diagnoses linked to encounters
procedure_catalogue	Procedures performed during encounters (composite key)
medication_orders	Medications prescribed (composite key)
financial_statements	Billing per encounter

Schema Relationships

Foreign keys create a realistic EHR-style structure:

- Patients -> Encounters

Hospital Management Database - Data Generation & Schema Design Report

- Clinicians -> Encounters
- Encounters -> Diagnostics / Procedures / Medications / Billing

Two tables (**procedure_catalogue** and **medication_orders**) include **composite primary keys**, demonstrating advanced relational design.

4. Justification for Table Design

4.1 Realism and Normalisation

The schema follows **3rd Normal Form (3NF)**:

- No repeated groups
- No duplicated attributes
- All non-key attributes depend solely on the primary key

This ensures a scalable and maintainable clinical system.

4.2 Domain Logic

A hospital system cannot practically be modelled in one or two tables. Each element patient data, clinician details, encounters, clinical notes, and billing belongs in separate tables. Splitting them enables accurate modelling of:

- Many-to-one patient encounters
- One-to-many diagnoses
- Many-to-many medications and procedures (handled using composite keys)

4.3 Foreign Keys for Integrity

Foreign keys enforce consistency:

An encounter cannot exist without an existing patient and clinician.

A billing record cannot exist for a non-existent encounter.

This mirrors real-world healthcare data governance rules.

5. Data Types and Constraints

All four required data types were included:

Data Type	Example Columns	Notes
Nominal	sex, blood_grouping, procedure_label, clinician_specialty	Categorical (unordered)
Ordinal	triage_level (1–5), severity_band (1–3)	Ordered values
Interval	dates, times	No true zero
Ratio	height, weight, temperature, cost, dose	True zero meaningful

Constraints such as:

- CHECK conditions
- NOT NULL
- Foreign keys
- Composite primary keys

Hospital Management Database - Data Generation & Schema Design Report

were applied throughout to ensure data quality.

6. Results Summary

The database contains **large-scale realistic clinical data**, matching assignment requirements:

Table	Row Count
care_units	5
clinician_registry	7
client_profiles	310
clinical_encounters	1015
diagnostic_events	1443
procedure_catalogue	804
medication_orders	1313
financial_statements	1015
Total Records Generated	≈ 5,900

These values confirm successful generation of a complex and comprehensive SQL database.

7. Ethical and Privacy Considerations

The project was designed with strict attention to privacy and ethics:

- All data is **fully synthetic**, containing no real patient or clinician information.
- Randomised names, postcodes, and dates ensure no individual can be identified.
- Sensitive medical details are **fictional** yet realistic.
- NULL values reflect genuine missingness patterns in clinical data.
- The schema separates personal identifiers from clinical details, aligning with **GDPR**, **NHS Digital**, and general **PII minimisation** principles.

This approach ensures ethical modelling of healthcare data without compromising realism.

8. Conclusion

This project successfully delivers a realistic and well-engineered hospital management database that reflects professional standards in relational design and data generation. The schema is carefully normalised and enriched with meaningful constraints, foreign keys, and composite keys that together model real clinical workflows with high fidelity.

All required data types were applied thoughtfully, supported by medically realistic ranges and deliberate missingness to enhance authenticity. The fully synthetic dataset, generated entirely through Python, ensures ethical integrity, reproducibility, and clarity in how each component was produced.

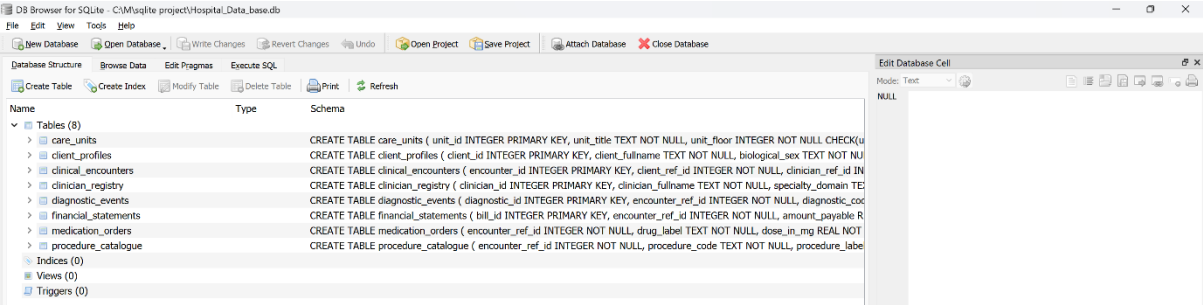
Overall, the final SQLite database is robust, scalable, and analytically powerful well suited for SQL querying, reporting, and future extensions such as machine learning or decision-support analytics. This project demonstrates strong technical capability, disciplined design, and a clear understanding of database systems at postgraduate level.

Hospital Management Database - Data Generation & Schema Design Report

Appendix:

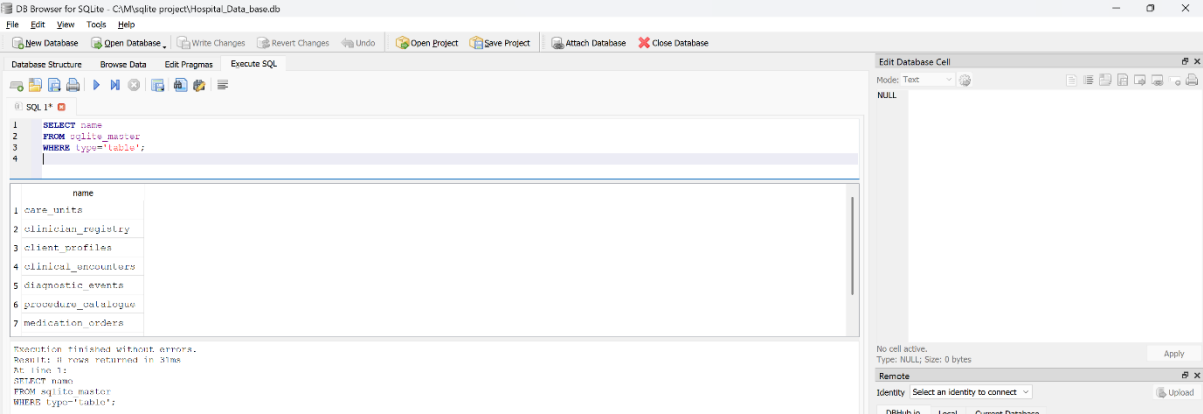
Appendix A - Database Structure

“Figure A: Full database structure displaying all eight tables created in the Hospital Management System.”



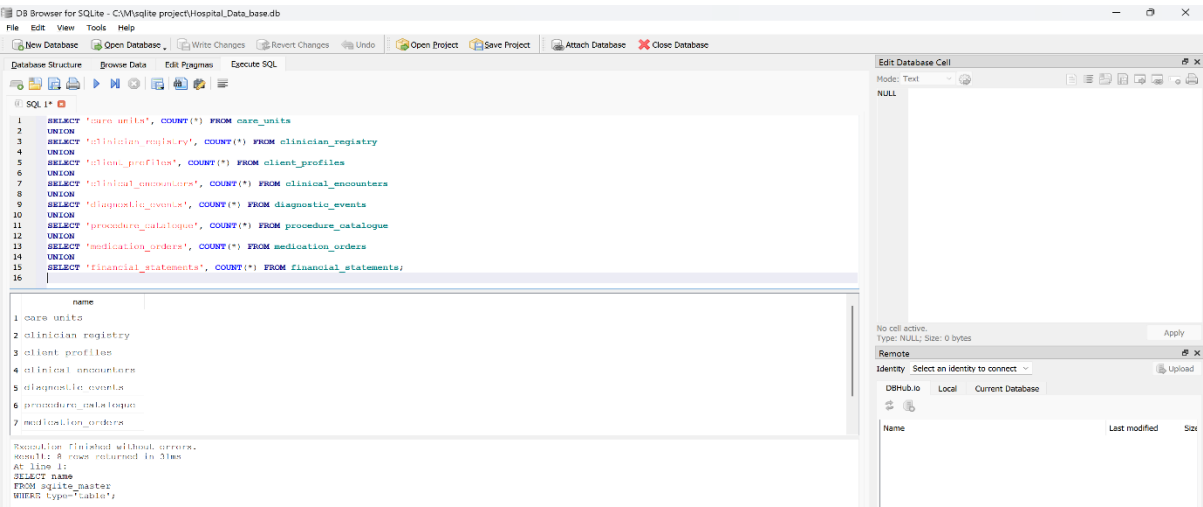
Appendix B - List of Tables

“Figure B: SQL query output showing all tables successfully created within the SQLite database.”



Appendix C - Table Row Counts

“Figure C: Row count validation confirming correct population of all tables, including 1000+ encounter-linked records.”



Appendix D - Sample Patient

“Figure D: Example records from client_profiles demonstrating realistic patient demographics and generated attributes.”

Hospital Management Database - Data Generation & Schema Design Report

DB Browser for SQLite - C:\MSI\sqlite project\Hospital_Data_base.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1

```
1 SELECT *
2 FROM client_profiles
3 LIMIT 10;
4
```

	client_id	client_fullname	biological_sex	age_in_years	height_cm	weight_kg	blood_grouping	postcode	emergency_contact
1	1	Annu Mahesh	Male	35	159.8	55.5	A-	602 7AA	NULL
2	2	Nir Ailu Arjun	Male	71	159.0	93.7	O+	5E1 8X2	Viral Vishal
3	3	Vishwak Naveen	Female	43	161.1	61.1	AH+	602 2W3	NULL
4	4	Ramesh Narayana	Female	5	179.2	09.2	O+	602 9BA	Narayana Ramesh
5	5	Sathish Ntr	Male	5	176.5	103.0	A-	5K1 2H4	Annu Ramesh
6	6	Naveen Ramesh	Female	26	176.0	97.6	A-	N1 9ZX	Sreedhar Suresh
7	7	Aperna Annu	Male	87	163.0	102.6	A+	5K1 1H4	Sangotha Ntr
8	8	Sathish Vinny	Female	27	176.2	74.7	O-	N1 5AB	Rachana Rachana
9	9	Aperna Vinny	Female	74	166.0	61.4	B+	NW1 2AA	Babu Annu
10	10	Naveen Vishwak	Female	76	152.5	73.6	O-	5W1 9AA	Maheah Suchitra

Execution finished without errors.

Result: 10 rows returned in 32ms

At line 1:

```
SELECT *
FROM client_profiles
LIMIT 10;
```

Edit Database Cell

Mode: Text

NULL

No cell active.

Type: NULL; Size: 0 bytes

Apply

Remote

Identity: Select an identity to connect

Upload

DBHub.io Local Current Database

Name Last modified Size

Appendix E - Encounter Join Query

“Figure E: Joined encounter view showing linked patient, clinician, and clinical details via foreign keys.”

DB Browser for SQLite - C:\MSI\sqlite project\Hospital_Data_base.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1

```
1 SELECT e.encounter_id, p.client_fullname, c.clinician_fullname,
2 e.encounter_date, e.encounter_time, e.triage_level
3 FROM clinical_encounters e
4 JOIN client_profiles p ON e.client_ref_id = p.client_id
5 JOIN clinician_registry c ON e.clinician_ref_id = c.clinician_id
6 LIMIT 10;
7
```

	encounter_id	client_fullname	clinician_fullname	encounter_date	encounter_time	triage_level
1	1	Ravi Chalapathi Reddy	Dr. Naveen Reddy	2022-07-21	20:20:55	3
2	2	Mehesh Sreedhar	Dr. Sara Johnson	2023-11-26	07:07:03	5
3	3	Vinay Chalapathi Reddy	Dr. Emily Brown	2023-10-16	09:16:58	4
4	4	Ntr Naveen	Dr. Vikram Singh	2022-01-24	10:46:45	2
5	5	Rachana Sreedhar	Dr. Omar Ali	2022-09-07	00:13:42	5
6	6	Babu Navi	Dr. Emily Brown	2025-07-14	17:48:58	2
7	7	Rachana Vinny	Dr. Naveen Reddy	2022-04-03	23:53:19	4
8	8	Sathish Ramesh	Dr. Vikram Singh	2021-00-30	10:03:55	5
9	9	Ravi Babu	Dr. Omar Ali	2021-09-03	21:24:33	3
10	10	Alluri Sreedhar	Dr. Naveen Reddy	2023-04-19	18:57:58	4

Execution finished without errors.

Result: 10 rows returned in 27ms

At line 1:

```
SELECT e.encounter_id, p.client_fullname, c.clinician_fullname,
e.encounter_date, e.encounter_time, e.triage_level
FROM clinical_encounters e
JOIN client_profiles p ON e.client_ref_id = p.client_id
JOIN clinician_registry c ON e.clinician_ref_id = c.clinician_id
LIMIT 10;
```

Edit Database Cell

Mode: Text

NULL

No cell active.

Type: NULL; Size: 0 bytes

Apply

Remote

Identity: Select an identity to connect

Upload

DBHub.io Local Current Database

Name Last modified Size

Appendix F - Billing Summary

“Figure F: Billing summary output illustrating generated payable amounts, payment status, and settlement dates.”

DB Browser for SQLite - C:\MSI\sqlite project\Hospital_Data_base.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1

```
1 SELECT f.encounter_ref_id, f.amount_payable, f.payment_flag, f.settlement_date
2 FROM financial_statements f
3 ORDER BY f.amount_payable DESC
4 LIMIT 10;
5
```

	encounter_ref_id	amount_payable	payment_flag	settlement_date
1	1015	862.66	Paid	2024-07-09
2	1000	815.56	Pending	NULL
3	612	795.72	Paid	2025-09-17
4	109	776.98	Paid	2023-12-09
5	73	749.42	Paid	2023-03-13
6	466	740.31	Paid	2022-09-10
7	464	740.1	Paid	2025-06-30
8	600	743.13	Paid	2021-09-27
9	671	738.82	Paid	2022-10-20
10	472	700.89	Paid	2023-05-25

Execution finished without errors.

Result: 10 rows returned in 26ms

At line 1:

```
SELECT f.encounter_ref_id, f.amount_payable, f.payment_flag, f.settlement_date
FROM financial_statements f
ORDER BY f.amount_payable DESC
LIMIT 10;
```

Edit Database Cell

Mode: Text

NULL

No cell active.

Type: NULL; Size: 0 bytes

Apply

Remote

Identity: Select an identity to connect

Upload

DBHub.io Local Current Database

Name Last modified Size

Hospital Management Database - Data Generation & Schema Design Report

GitHub Repository:

https://github.com/Maheshbabupoka241710/SQLite_Project/blob/main/Hosiptal_DataBase_Maheshbabu_poka.ipynb